UNIVERSITY OF PORTSMOUTH

COURSEWORK 2

SUBJECT-DATA MANAGEMENT

Assessment 2 - FM CLOTHING STORE case study using Mongo DB

SUBMITTED BY:

UP2290492

UP2302988

Table of Content

Introduction

The objective of this project is to develop a robust MongoDB database for FM Clothing Store, designed to streamline key business processes and provide actionable insights into performance metrics. The database is structured to include core entities such as **Customers**, **Orders**,

Products, **Staff**, **Stores**, and **Categories**, all of which are critical to the store's operations. By leveraging MongoDB's adaptability and scalability, the project showcases how NoSQL databases can address practical business requirements effectively.

The project tasks include creating a structured database schema and formulating queries to achieve specific business objectives. These objectives encompass retrieving detailed order data, calculating store revenue by product and category, identifying customers who purchased across multiple categories, and summarizing key performance metrics like total orders, revenue, and unique customers. MongoDB's aggregation framework plays a vital role in processing and presenting data in a meaningful and actionable manner.

Throughout the project, we explored MongoDB's advanced features, including the use of embedded documents and references for efficient data modeling, aggregation pipelines for complex data processing, and indexing to optimize query performance. Challenges such as managing relationships in a NoSQL environment and constructing multi-stage queries required a strong grasp of both technical concepts and business logic, fostering critical thinking and problem-solving skills.

The outcomes of this project underscore the significance of well-structured databases in enhancing business efficiency and generating valuable insights. Through this coursework, we not only developed a practical database solution tailored to business needs but also honed essential skills such as collaboration, analytical thinking, and technical expertise. The subsequent sections delve into the database design, query development, and the key takeaways from this learning experience.

Task 1

Database Development

- Customer

```
fm_store> db.Customers.insertMany([
...     { "_id": 1, "fname": "John", "lname": "Doe", "address": "123 Fashion St", "phone": "123-456-7890", "email": "john.doe@example.com" },
...     { "_id": 2, "fname": "Jane", "lname": "Smith", "address": "456 Trendy Ave", "phone": "987-654-3210", "email": "jane.smith@example.com" },
...     { "_id": 3, "fname": "Alice", "lname": "Brown", "address": "789 Chic Blvd", "phone": "555-666-7777", "email": "alice.brown@example.com" },
...     { "_id": 4, "fname": "Bob", "lname": "Johnson", "address": "101 Urban Rd", "phone": "444-555-6666", "email": "bob.johnson@example.com" },
...     { "_id": 5, "fname": "Charlie", "lname": "Davis", "address": "202 Designer Ln", "phone": "111-222-3333", "email": "charlie.davis@example.com" },
...     { "_id": 6, "fname": "Emily", "lname": "Clark", "address": "303 Couture Ct", "phone": "333-444-5555", "email": "emily.clark@example.com" },
...     { "_id": 7, "fname": "Frank", "lname": "Lee", "address": "404 Style St", "phone": "777-888-9999", "email": "frank.lee@example.com" },
...     { "_id": 8, "fname": "Grace", "lname": "Walker", "address": "505 Vogue Dr", "phone": "222-333-4444", "email": "grace.walker@example.com" },
...     { "_id": 9, "fname": "Henry", "lname": "Young", "address": "606 Runway Ave", "phone": "888-999-1111", "email": "henry.young@example.com" },
...     { "_id": 10, "fname": "Ivy", "lname": "Green", "address": "707 Elegant St", "phone": "666-777-8888", "email": "ivy.green@example.com" }
... ]);
{
  acknowledged: true,
  insertedIds: {
    '0': 1,
    '1': 2,
    '2': 3,
    '3': 4,
    '4': 5,
    '5': 6,
    '6': 7,
    '7': 8,
    '8': 9,
    '9': 10
  }
}
```

- Orders

```
fm_store> db.Orders.insertMany([
...     { "_id": 1, "customer_id": 1, "order_date": "2025-01-10", "delivery_address": "123 Fashion St", "status": "Delivered", "total_amount": 59.98 },
...     { "_id": 2, "customer_id": 2, "order_date": "2025-01-11", "delivery_address": "456 Trendy Ave", "status": "Pending", "total_amount": 89.99 },
...     { "_id": 3, "customer_id": 3, "order_date": "2025-01-12", "delivery_address": "789 Chic Blvd", "status": "Canceled", "total_amount": 129.99 },
...     { "_id": 4, "customer_id": 4, "order_date": "2025-01-13", "delivery_address": "101 Urban Rd", "status": "Delivered", "total_amount": 34.99 },
...     { "_id": 5, "customer_id": 5, "order_date": "2025-01-14", "delivery_address": "202 Designer Ln", "status": "Pending", "total_amount": 49.99 },
...     { "_id": 6, "customer_id": 6, "order_date": "2025-01-15", "delivery_address": "303 Couture Ct", "status": "Delivered", "total_amount": 19.99 },
...     { "_id": 7, "customer_id": 7, "order_date": "2025-01-16", "delivery_address": "404 Style St", "status": "Canceled", "total_amount": 39.99 },
...     { "_id": 8, "customer_id": 8, "order_date": "2025-01-17", "delivery_address": "505 Vogue Dr", "status": "Pending", "total_amount": 24.99 },
...     { "_id": 9, "customer_id": 9, "order_date": "2025-01-18", "delivery_address": "606 Runway Ave", "status": "Delivered", "total_amount": 89.99 },
...     { "_id": 10, "customer_id": 10, "order_date": "2025-01-19", "delivery_address": "707 Elegant St", "status": "Pending", "total_amount": 29.99 }
... ]);
{
  acknowledged: true,
  insertedIds: {
    '0': 1,
    '1': 2,
    '2': 3,
    '3': 4,
    '4': 5,
    '5': 6,
    '6': 7,
    '7': 8,
    '8': 9,
    '9': 10
  }
}
```

- Order Details

```
fm_store> db.OrderDetails.insertMany([
...     { "_id": 1, "order_id": 1, "product_id": 1, "quantity": 2, "price": 29.99 },
...     { "_id": 2, "order_id": 2, "product_id": 5, "quantity": 1, "price": 89.99 },
...     { "_id": 3, "order_id": 3, "product_id": 7, "quantity": 1, "price": 129.99 },
...     { "_id": 4, "order_id": 4, "product_id": 6, "quantity": 1, "price": 34.99 },
...     { "_id": 5, "order_id": 5, "product_id": 2, "quantity": 1, "price": 49.99 },
...     { "_id": 6, "order_id": 6, "product_id": 9, "quantity": 1, "price": 19.99 },
...     { "_id": 7, "order_id": 7, "product_id": 4, "quantity": 1, "price": 39.99 },
...     { "_id": 8, "order_id": 8, "product_id": 8, "quantity": 1, "price": 24.99 },
...     { "_id": 9, "order_id": 9, "product_id": 5, "quantity": 1, "price": 89.99 },
...     { "_id": 10, "order_id": 10, "product_id": 10, "quantity": 1, "price": 29.99 }
... ]);
{
  acknowledged: true,
  insertedIds: {
    '0': 1,
    '1': 2,
    '2': 3,
    '3': 4,
    '4': 5,
    '5': 6,
    '6': 7,
    '7': 8,
    '8': 9,
    '9': 10
  }
}
```

- Products

```
  }
}
fm_store> db.Products.insertMany([
...     { "_id": 1, "name": "Men's Cotton Shirt", "description": "Classic button-up cotton shirt", "size": "M", "composition": "100% Cotton", "price": 29.99, "category_id": 1, "stock_count": 40 },
...     { "_id": 2, "name": "Women's Maxi Dress", "description": "Floral long dress", "size": "L", "composition": "Polyester", "price": 49.99, "category_id": 2, "stock_count": 25 },
...     { "_id": 3, "name": "Leather Belt", "description": "Genuine leather belt", "size": "Adjustable", "composition": "Leather", "price": 19.99, "category_id": 3, "stock_count": 50 },
...     { "_id": 4, "name": "Men's Sneakers", "description": "Comfortable casual sneakers", "size": "10", "composition": "Rubber, Fabric", "price": 39.99, "category_id": 4, "stock_count": 30 },
...     { "_id": 5, "name": "Women's Trench Coat", "description": "Classic long trench coat", "size": "M", "composition": "Wool-Poly Blend", "price": 89.99, "category_id": 5, "stock_count": 15 },
...     { "_id": 6, "name": "Hoodie", "description": "Casual unisex hoodie", "size": "L", "composition": "Fleece", "price": 34.99, "category_id": 6, "stock_count": 20 },
...     { "_id": 7, "name": "Men's Suit", "description": "Two-piece formal suit", "size": "42", "composition": "Wool Blend", "price": 129.99, "category_id": 7, "stock_count": 10 },
...     { "_id": 8, "name": "Running Shorts", "description": "Lightweight shorts for sports", "size": "M", "composition": "Polyester", "price": 24.99, "category_id": 8, "stock_count": 40 },
...     { "_id": 9, "name": "Winter Scarf", "description": "Warm knit scarf", "size": "One Size", "composition": "Wool", "price": 19.99, "category_id": 9, "stock_count": 30 },
...     { "_id": 10, "name": "Women's Pajama Set", "description": "Soft cotton loungewear set", "size": "S", "composition": "100% Cotton", "price": 29.99, "category_id": 10, "stock_count": 20 }
... ]);
{
  acknowledged: true,
  insertedIds: {
    '0': 1,
    '1': 2,
    '2': 3,
    '3': 4,
    '4': 5,
    '5': 6,
    '6': 7,
    '7': 8,
    '8': 9,
    '9': 10
  }
}
```

- Staff

```
fm_store> db.Staff.insertMany([
...     { "_id": 1, "name": "Alice", "position": "Manager", "store_id": 1, "shift_pattern": "9am-5pm" },
...     { "_id": 2, "name": "Bob", "position": "Cashier", "store_id": 2, "shift_pattern": "10am-6pm" },
...     { "_id": 3, "name": "Charlie", "position": "Clerk", "store_id": 3, "shift_pattern": "12pm-8pm" },
...     { "_id": 4, "name": "David", "position": "Security", "store_id": 4, "shift_pattern": "8am-4pm" },
...     { "_id": 5, "name": "Eve", "position": "Cleaner", "store_id": 5, "shift_pattern": "6am-2pm" },
...     { "_id": 6, "name": "Frank", "position": "Assistant Manager", "store_id": 1, "shift_pattern": "10am-6pm" },
...     { "_id": 7, "name": "Grace", "position": "Stock Manager", "store_id": 2, "shift_pattern": "9am-5pm" },
...     { "_id": 8, "name": "Henry", "position": "Sales Representative", "store_id": 3, "shift_pattern": "11am-7pm" },
...     { "_id": 9, "name": "Ivy", "position": "Visual Merchandiser", "store_id": 4, "shift_pattern": "10am-6pm" },
...     { "_id": 10, "name": "Jack", "position": "Customer Support", "store_id": 5, "shift_pattern": "9am-5pm" }
... ]);
{
  acknowledged: true,
  insertedIds: {
    '0': 1,
    '1': 2,
    '2': 3,
    '3': 4,
    '4': 5,
    '5': 6,
    '6': 7,
    '7': 8,
    '8': 9,
    '9': 10
  }
}
```

- Stores

```
fm_store> db.Stores.insertMany([
...     { "_id": 1, "store_location": "Portsmouth" },
...     { "_id": 2, "store_location": "Chichester" },
...     { "_id": 3, "store_location": "Havant" },
...     { "_id": 4, "store_location": "Fareham" },
...     { "_id": 5, "store_location": "Waterlooville" }
... ]);
{
  acknowledged: true,
  insertedIds: { '0': 1, '1': 2, '2': 3, '3': 4, '4': 5 }
}
```

- Categories

```
fm_store> db.Categories.insertMany([
...     { "_id": 1, "category_name": "Men's Clothing" },
...     { "_id": 2, "category_name": "Women's Clothing" },
...     { "_id": 3, "category_name": "Accessories" },
...     { "_id": 4, "category_name": "Footwear" },
...     { "_id": 5, "category_name": "Outerwear" },
...     { "_id": 6, "category_name": "Casual Wear" },
...     { "_id": 7, "category_name": "Formal Wear" },
...     { "_id": 8, "category_name": "Sportswear" },
...     { "_id": 9, "category_name": "Seasonal Clothing" },
...     { "_id": 10, "category_name": "Loungewear" }
... ]);
{
  acknowledged: true,
  insertedIds: {
    '0': 1,
    '1': 2,
    '2': 3,
    '3': 4,
    '4': 5,
    '5': 6,
    '6': 7,
    '7': 8,
    '8': 9,
    '9': 10
  }
}
```

Count Function

To show every detail in our collections, we used the count function.

```
fm_store> db.Categories.countDocuments();
30
fm_store>

fm_store> db.Customers.countDocuments();
30
fm_store>

fm_store> db.OrderDetails.countDocuments();
30
fm_store>

fm_store> db.Orders.countDocuments();
30
fm_store>

fm_store> db.Products.countDocuments();
100
fm_store>

fm_store> db.Staff.countDocuments();
30
fm_store>

fm_store> db.Stores.countDocuments();
5
```

Task 2

Queries

1. **List all orders with customer, product, and category details**

```
fm_store> db.OrderDetails.aggregate([
...     {
.....         $lookup: {
.......             from: "Orders",
.......             localField: "order_id",
.......             foreignField: "_id",
.......             as: "order_details"
.......         }
.....     },
...     {
.....         $lookup: {
.......             from: "Customers",
.......             localField: "order_details.customer_id",
.......             foreignField: "_id",
.......             as: "customer_details"
.......         }
.....     },
...     {
.....         $lookup: {
.......             from: "Products",
.......             localField: "product_id",
.......             foreignField: "_id",
.......             as: "product_details"
.......         }
.....     },
...     {
.....         $lookup: {
.......             from: "Categories",
.......             localField: "product_details.category_id",
.......             foreignField: "_id",
.......             as: "category_details"
.......         }
.....     }
... ]);
[
  {
    _id: 1,
    order_id: 1,
    product_id: 1,
    quantity: 2,
    price: 29.99,
    order_details: [
      {
        _id: 1,
        customer_id: 1,
        order_date: '2025-01-10',
        delivery_address: '123 Fashion St',
        status: 'Delivered',
        total_amount: 59.98
      }
    ],
    customer_details: [
      {
        _id: 1,
        fname: 'John',
        lname: 'Doe',
        address: '123 Fashion St',
        phone: '123-456-7890',
        email: 'john.doe@example.com'
      }
    ],
    product_details: [
      {
        _id: 1,
        name: "Men's Cotton Shirt",
        description: 'Classic button-up cotton shirt',
        size: 'M',
        composition: '100% Cotton',
        price: 29.99,
        category_id: 1,
        stock_count: 40
      }
    ],
    category_details: [ { _id: 1, category_name: "Men's Clothing" } ]
  },
  {
    _id: 2,
    order_id: 2,
    product_id: 5,
    quantity: 1,
    price: 89.99,
    order_details: [
      {
        _id: 2,
        customer_id: 2,
        order_date: '2025-01-11',
        delivery_address: '456 Trendy Ave',
        status: 'Pending',
        total_amount: 89.99
      }
    ],
    customer_details: [
      {
```

.....

.....

.....

Objective:

To retrieve comprehensive details of all orders, including:

- Customer details (name, email, etc.).
- Product details (name, description, price, etc.).
- Product category information.

Steps:

- **Join OrderDetails with Orders:** Links order details (e.g., product and quantity) with the corresponding order.
- **Join Customers:** Adds customer information to each order.
- **Join Products:** Retrieves product details for each order.
- **Join Categories:** Links each product with its category information.
- **Project:** Includes selected fields (e.g., order ID, product details, customer details).

Result:

- A complete view of all orders, showing the ordered products, customer information, and product categories.

2. Total revenue for each store with product and category breakdown

```
fm_store> db.Staff.aggregate([
...     {
.....         $lookup: {
.......               from: "Orders",
.......               localField: "store_id",
.......               foreignField: "customer_id",
.......               as: "store_orders"
.......           }
.....     },
...     {
.....         $unwind: "$store_orders"
.....     },
...     {
.....         $lookup: {
.......               from: "OrderDetails",
.......               localField: "store_orders._id",
.......               foreignField: "order_id",
.......               as: "order_details"
.......           }
.....     },
...     {
.....         $unwind: "$order_details"
.....     },
...     {
.....         $lookup: {
.......               from: "Products",
.......               localField: "order_details.product_id",
.......               foreignField: "_id",
.......               as: "product_details"
.......           }
.....     },
...     {
.....         $unwind: "$product_details"
.....     },
...     {
.....         $lookup: {
.......               from: "Categories",
.......               localField: "product_details.category_id",
.......               foreignField: "_id",
.......               as: "category_details"
.......           }
.....     },
...     {
.....         $group: {
.......               _id: "$store_id",
.......               total_revenue: { $sum: { $multiply: ["$order_details.price", "$order_details.quantity"] } },
.......               product_breakdown: { $push: "$product_details.name" },
.......               category_breakdown: { $push: "$category_details.category_name" }
.......           }
.....     }
... ]);
[
  {
    _id: 1,
    total_revenue: 119.96,
    product_breakdown: [ "Men's Cotton Shirt", "Men's Cotton Shirt" ],
    category_breakdown: [ [ "Men's Clothing" ], [ "Men's Clothing" ] ]
  },
  {
    _id: 4,
    total_revenue: 69.98,
    product_breakdown: [ 'Hoodie', 'Hoodie' ],
    category_breakdown: [ [ 'Casual Wear' ], [ 'Casual Wear' ] ]
  },
  {
    _id: 5,
    total_revenue: 99.98,
    product_breakdown: [ "Women's Maxi Dress", "Women's Maxi Dress" ],
    category_breakdown: [ [ "Women's Clothing" ], [ "Women's Clothing" ] ]
  },
  {
    _id: 2,
    total_revenue: 179.98,
    product_breakdown: [ "Women's Trench Coat", "Women's Trench Coat" ],
    category_breakdown: [ [ 'Outerwear' ], [ 'Outerwear' ] ]
  },
  {
    _id: 3,
    total_revenue: 259.98,
    product_breakdown: [ "Men's Suit", "Men's Suit" ],
    category_breakdown: [ [ 'Formal Wear' ], [ 'Formal Wear' ] ]
  }
]
```

Objective:

To calculate total revenue for each store and display a breakdown of sold products and their categories.

Steps:

- **Join Staff with Orders:** Links store staff with the orders they processed.
- **Join OrderDetails:** Adds order details (e.g., product and quantity) to each order.
- **Join Products:** Includes product information for each order.
- **Join Categories:** Links products to their categories.
- **Group by Store ID:**
  - Calculates total revenue ($sum).
  - Lists all sold products and their categories.
- **Result Sorting:** Outputs results grouped by store.

Result:

- Each store's total revenue and a detailed list of products and categories contributing to the revenue.

3. Find customers who ordered products from multiple categorie

```
fm_store> db.OrderDetails.aggregate([
...     {
.....         $lookup: {
.......             from: "Products",
.......             localField: "product_id",
.......             foreignField: "_id",
.......             as: "product_details"
.......         }
.....     },
...     {
.....         $unwind: "$product_details"
.....     },
...     {
.....         $lookup: {
.......             from: "Categories",
.......             localField: "product_details.category_id",
.......             foreignField: "_id",
.......             as: "category_details"
.......         }
.....     },
...     {
.....         $unwind: "$category_details"
.....     },
...     {
.....         $lookup: {
.......             from: "Orders",
.......             localField: "order_id",
.......             foreignField: "_id",
.......             as: "order_details"
.......         }
.....     },
...     {
.....         $unwind: "$order_details"
.....     },
...     {
.....         $lookup: {
.......             from: "Customers",
.......             localField: "order_details.customer_id",
.......             foreignField: "_id",
.......             as: "customer_details"
.......         }
.....     },
...     {
.....         $unwind: "$customer_details"
.....     },
...     {
.....         $group: {
.......             _id: {
.........                 category_name: "$category_details.category_name",
.........                 customer_id: "$customer_details._id"
.........             },
.......             max_price: { $max: "$product_details.price" },
.......             customer_name: { $first: "$customer_details.fname" },
.......             customer_email: { $first: "$customer_details.email" },
.......             product_name: { $first: "$product_details.name" }
.......         }
.....     },
...     {
.....         $sort: { "_id.category_name": 1, "max_price": -1 }
.....     }
... ]);
[
  {
    _id: { category_name: 'Accessories', customer_id: 12 },
    max_price: 19.99,
    customer_name: 'Linda',
    customer_email: 'linda.martinez@example.com',
    product_name: 'Leather Belt'
  },
  {
    _id: { category_name: 'Accessories', customer_id: 22 },
    max_price: 19.99,
    customer_name: 'Jackson',
    customer_email: 'jackson.white@example.com',
    product_name: 'Leather Belt'
  },
  {
    _id: { category_name: 'Casual Wear', customer_id: 20 },
    max_price: 34.99,
    customer_name: 'Emma',
    customer_email: 'emma.scott@example.com',
    product_name: 'Hoodie'
  },
  {
    _id: { category_name: 'Casual Wear', customer_id: 30 },
    max_price: 34.99,
    customer_name: 'Chloe',
    customer_email: 'chloe.carter@example.com',
    product_name: 'Hoodie'
  },
  {
    _id: { category_name: 'Casual Wear', customer_id: 4 },
    max_price: 34.99,
```

Objective:

To identify customers who purchased products from more than one category.

Steps:

- **Join Orders with OrderDetails:** Links orders with the products in each order.
- **Join Products:** Retrieves product details for each order.
- **Join Categories:** Adds category information for the products.
- **Group by Customer ID:**
  - Creates a list of unique categories for each customer ($addToSet).
- **Filter Customers:** Retains only customers with more than one category in their list ("categories.1": { $exists: true }).
- **Join Customers:** Adds customer details (name, email, etc.).

Result:

- A list of customers who purchased products from multiple categories, with their details and purchased categories.

4. List products sold by each store with category and order details

```
Type "it" for more
fm_store> db.Staff.aggregate([
...     {
.....         $lookup: {
.......             from: "Orders",
.......             localField: "store_id",
.......             foreignField: "customer_id",
.......             as: "store_orders"
.......           }
.....     },
...     {
.....         $unwind: "$store_orders"
.....     },
...     {
.....         $lookup: {
.......             from: "OrderDetails",
.......             localField: "store_orders._id",
.......             foreignField: "order_id",
.......             as: "order_details"
.......           }
.....     },
...     {
.....         $unwind: "$order_details"
.....     },
...     {
.....         $lookup: {
.......             from: "Products",
.......             localField: "order_details.product_id",
.......             foreignField: "_id",
.......             as: "product_details"
.......           }
.....     },
...     {
.....         $unwind: "$product_details"
.....     },
...     {
.....         $lookup: {
.......             from: "Categories",
.......             localField: "product_details.category_id",
.......             foreignField: "_id",
.......             as: "category_details"
.......           }
.....     },
...     {
.....         $group: {
.......             _id: "$store_id",
.......             products: { $push: { name: "$product_details.name", category: "$category_details.category_name" } }
.......           }
.....     }
... ]);
[
  {
    _id: 1,
    products: [
      { name: "Men's Cotton Shirt", category: [ "Men's Clothing" ] },
      { name: "Men's Cotton Shirt", category: [ "Men's Clothing" ] }
    ]
  },
  {
    _id: 4,
    products: [
      { name: 'Hoodie', category: [ 'Casual Wear' ] },
      { name: 'Hoodie', category: [ 'Casual Wear' ] }
    ]
  },
  {
    _id: 5,
    products: [
      { name: "Women's Maxi Dress", category: [ "Women's Clothing" ] },
      { name: "Women's Maxi Dress", category: [ "Women's Clothing" ] }
    ]
  },
  {
    _id: 2,
    products: [
      { name: "Women's Trench Coat", category: [ 'Outerwear' ] },
      { name: "Women's Trench Coat", category: [ 'Outerwear' ] }
    ]
  },
  {
    _id: 3,
    products: [
      { name: "Men's Suit", category: [ 'Formal Wear' ] },
      { name: "Men's Suit", category: [ 'Formal Wear' ] }
    ]
  }
]
```

Objective:

To find all products sold by each store and provide details about the products and their categories.

Steps:

- **Join Staff with Orders:** Connects staff members to the orders they processed.
- **Join OrderDetails:** Adds order details (e.g., product and quantity).
- **Join Products:** Retrieves product information for each order.
- **Join Categories:** Links products to their categories.
- **Group by Store ID:**
  - Creates a list of products sold by each store.
  - Includes the product name and category for each sold item.

Result:

- Each store with a list of sold products and their corresponding categories.

5. Total orders, revenue, and unique customers per store

```
fm_store> db.Staff.aggregate([
...     {
.....         $lookup: {
.......             from: "Orders",
.......             localField: "store_id",
.......             foreignField: "customer_id",
.......             as: "store_orders"
.......           }
.....     },
...     {
.....         $unwind: "$store_orders"
.....     },
...     {
.....         $lookup: {
.......             from: "OrderDetails",
.......             localField: "store_orders._id",
.......             foreignField: "order_id",
.......             as: "order_details"
.......           }
.....     },
...     {
.....         $unwind: "$order_details"
.....     },
...     {
.....         $group: {
.......             _id: "$store_id",
.......             total_orders: { $sum: 1 },
.......             total_revenue: { $sum: { $multiply: ["$order_details.price", "$order_details.quantity"] } },
.......             unique_customers: { $addToSet: "$store_orders.customer_id" }
.......           }
.....     },
...     {
.....         $project: {
.......             total_orders: 1,
.......             total_revenue: 1,
.......             unique_customer_count: { $size: "$unique_customers" }
.......           }
.....     }
... ]);
[
  {
    _id: 1,
    total_orders: 2,
    total_revenue: 119.96,
    unique_customer_count: 1
  },
  {
    _id: 4,
    total_orders: 2,
    total_revenue: 69.98,
    unique_customer_count: 1
  },
  {
    _id: 5,
    total_orders: 2,
    total_revenue: 99.98,
    unique_customer_count: 1
  },
  {
    _id: 2,
    total_orders: 2,
    total_revenue: 179.98,
    unique_customer_count: 1
  },
  {
    _id: 3,
    total_orders: 2,
    total_revenue: 259.98,
    unique_customer_count: 1
  }
]
```

Objective:

To calculate total orders, revenue, and the number of unique customers for each store.

Steps:

- **Join Staff with Orders:** Links staff members to the orders they processed.
- **Join OrderDetails:** Adds product and quantity details to each order.
- **Group by Store ID:**
  - Counts total orders ($sum: 1).
  - Calculates total revenue ($sum: { $multiply: ["$price", "$quantity"] }).
  - Collects unique customer IDs for each store ($addToSet).

- **Project Results:**
  Outputs total orders, total revenue, and the count of unique customers.

Result:

- Each store with:
  - Total number of orders.
  - Total revenue generated.
  - The number of unique customers served.

Task 4

Reflective Report

Creating and configuring a database:

We took idea from previous PgSQL database and Design the Database for FM Clothing Store , the process of recognizing the entities of our application (Customers, Orders, Products, Staff, Stores and Categories) and understanding their relationships with each other gave us a better understanding of our database. We showcased MongoDB's ability to handle hierarchical data in a wide variety of ways, illustrating both the embedding and linking approach to establish relationships.

Optimizing the query

Writing complex queries using aggregation pipeline to do things such as calculating revenue, finding consumers who purchase from multiple groups, and giving provide store-specific product data was a major highlight. These exercises show the importance of efficiently ordering queries instead of pulling useful information from large data sets.

Exercise critical thought:

It was important to know exactly what the company wanted for the job. For example, you needed not only technical know-how, but also sound business logic skills to solve how to decompose revenue or how to get clients that ordered from multiple categories.

Cooperation and teamwork:

Working as a team made us better team workers. The regular collaboration and communication were necessary to spread the work like writing the queries, creating documentation, and configuring the database. Often the responses from peer discussions were better.

The exercise showed the importance of tackling problems in a systematic way. Starting with a good understanding of what we wanted to implement — and then implementing and testing little pieces bit by bit worked really well. Not only did reviewing each other's work help improve it as a team, we learned from each other in the process. This task enabled us to grow our technical knowledge and better understand the practical purpose of database management. Combining our commercial and technical know-how we made FM Clothing Store a really exciting response.

References

Dr. Matthew, O. (n.d.). Data management [Course module]. Moodle. University of Portsmouth. Retrieved January 22, 2025, from https://moodle.port.ac.uk/course/view.php?id=25672

W3Schools. (n.d.). MongoDB tutorial. W3Schools. Retrieved January 22, 2025, from https://www.w3schools.com/mongodb/

MongoDB, Inc. (n.d.). MongoDB Atlas documentation. MongoDB. Retrieved January 22, 2025, from https://www.mongodb.com/docs/atlas/